Universidad de Las Palmas de Gran Canaria (ULPGC)
Escuela de Ingeniería Informática
Grado en Ciencia e Ingeniería de Datos — Big Data (Academic Year 2025–2026)

Stage 2: Building a Service-Oriented Architecture

Search Engine Project — Cooperative Microservices, Hybrid Event-Driven Coordination, and Empirical Performance Evaluation

Group Name: 4Bytees
Team Members: Lucas Rodríguez Hernández (lucas.rodriguez104@alu.ulpgc.es)
Javier Bolívar García-Izquierdo (javer.bolivar101@alu.ulpgc.es)
June Argoitia Aguirrezabalaga (june.argoitia101@alu.ulpgc.es)
Sara Dévora Ortega (sara.devora103@alu.ulpgc.es)
Course & Stage: Big Data — Stage 2: Single-Host Service-Oriented Architecture (SOA)
GitHub Repository: https://github.com/4Bytees/stage_2
Technology Stack: Java 17, Javalin 6.1.3, ActiveMQ Classic 5.18.3, Hazelcast 5.3.6, SQLite 3.45.1, Nginx

Abstract. This technical report documents the architectural design, microservice implementation, failure-recovery mechanisms, and experimental benchmarking of the **Stage 2 Service-Oriented Architecture (SOA)** for the Project Gutenberg Search Engine developed by Group **FourByTees** (`com.fourbytees`) [1]. Evolving from the monolithic Stage 1 prototype [2], the system is decomposed into three containerized Java 17 microservices—`ingestion-service`, `indexing-service`, and `search-service`—deployed on a single host via Docker Compose [9]. The architecture combines synchronous HTTP/REST contracts (documented in OpenAPI 3.0.3 [11]) behind an Nginx reverse proxy [10] with asynchronous point-to-point JMS queues in Apache ActiveMQ Classic 5.18.3 [6] and distributed coordination structures in Hazelcast 5.3.6 [5]. We present verifiable solutions to four distributed systems challenges: (i) crash-consistent dual Transactional Outbox pipelines (`OutboxService` in ingestion and `BookIndexedPublisher` in indexing) [13], (ii) lease-based per-book claim coordination (`BookClaim`) with automatic stale-claim takeover and exponential JMS redelivery backoff, (iii) zero-downtime atomic full-index rebuilds (`IndexRebuilder`) via temporary SQLite database construction and transactional `ATTACH DATABASE` swapping [7], and (iv) strict client error sanitization (`ApiErrors`). Our experimental methodology defines exact hardware/JVM specifications, the 25-book multilingual corpus (15.57 MiB logical datalake size, 192,167 inverted-index postings), mathematical timing boundaries, and statistical distributions across JMH 1.37 microbenchmarks [8] and live Docker integration benchmarks [3].

Contents

1 Introduction and Objectives **2**
1.1 Stage 2 Objectives and Scope . . . 2

2 System Architecture **2**
2.1 Architectural Topology and Communication Boundaries . . . 2
2.2 Container Inventory, Resource Limits, and Concurrency Hierarchy . . . 3

3 API Design and Contracts **3**
3.1 Front-End-Facing HTTP API (`search-service` via Nginx on Port `7000`) . . . 3
3.2 Internal and Operational Service HTTP APIs . . . 4
3.3 Non-HTTP Asynchronous Messaging and Distributed Coordination Contracts . . . 4
3.4 Functional, End-to-End, and Resilience Validation Suite . . . 4

4 Microservice Implementation **6**
4.1 Code Hygiene and Benchmark Isolation (Addressing Stage 1 Feedback #41) . . . 6
4.2 Shared Module (`common-dto`) . . . 6
4.3 Ingestion Service (`ingestion-service`) . . . 6
4.4 Indexing Service (`indexing-service`) . . . 7
4.5 Search Service (`search-service`) . . . 7

5 Design Decisions **8**
5.1 Hybrid Coordination (ActiveMQ + Hazelcast + Dual Outbox) vs. Centralized Orchestration . . . 8
5.2 Three-Tier Concurrency Model: Clients, Container Replicas, and Process Threads . . . 8
5.3 Shared State Protection: Datalake Atomicity and SQLite WAL Locking . . . 8
5.4 Comprehensive Quantitative Justification of Datalake, SQLite, and Hazelcast (Feedback #4, #10, #25, #27, #30) . . . 9
5.5 Verification of Idempotency, Crash Recovery, and Failure Consistency (Feedback #9, #32, #33, #40) 9

6 Experimental Methodology and Benchmarks **9**
6.1 Experimental Setup and Methodology . . . 10
6.1.1 Hardware, Operating System, JVM, and Container Environment . . . 10
6.1.2 Exact Input Dataset: The 25-Book Multilingual Seed Corpus . . . 10
6.1.3 Exact Queries Executed and Ground-Truth Verification (Feedback #28, #33) . . . 10
6.1.4 Mathematical Timing Boundaries (Feedback #2, #6, #14, #22) . . . 12
6.1.5 Preparation, Cache State, Warm-up, and Statistical Protocol (Feedback #37, #38) . . . 13
6.1.6 Traceability and Reproducibility Matrix (Feedback #11, #39) . . . 13
6.2 Component-Level Microbenchmarks (JMH) . . . 13
6.3 System-Level Integration Benchmarks . . . 15
6.3.1 Per-Book Indexing Latency and Full Atomic Rebuild (`indexing-book-latency.csv`) . . . 15
6.3.2 HTTP Search Throughput and Latency Under Client Concurrency (`http-search-concurrency.csv`) . . . 16
6.3.3 Container Resource Telemetry and Rigorous Memory Metric Definition (Feedback #20, #24, #29) . . . 16
6.4 Replication, Threading, and Bottleneck Analysis . . . 17
6.5 Limitations and Threats to Validity (Feedback #12, #13, #31, #34) . . . 17

7 Conclusions and Stage 3 Preparation **18**
7.1 Empirical Conclusions Scoped to the Stage 2 Single-Host Deployment . . . 18
7.2 Architectural Roadmap for Stage 3 (Multi-Node Cluster Deployment) . . . 18

1 Introduction and Objectives

In Stage 1 of the Big Data Search Engine project [2], document ingestion, metadata parsing, inverted-index construction, and search query execution were coupled within a single monolithic Java process. While effective for establishing the baseline data layer and frozen tokenization rules, the monolithic architecture exhibited four structural limitations when subjected to concurrent workloads:

1. **Shared Fate and Coupled Failure Domains:** Because acquisition, parsing, and query serving shared a single JVM heap and process lifecycle, a slow network download from Project Gutenberg [12] or an unhandled storage exception during indexing could stall or terminate the user-facing search service.
2. **Inability to Scale Read and Write Workloads Independently:** Full-text search queries are read-intensive and latency-sensitive, whereas inverted-index construction is write-intensive and serialized by SQLite's single-writer lock [7]. A monolithic deployment cannot scale stateless read replicas independently of background indexing workers.
3. **Synchronous Blocking Ingestion-to-Indexing Pipeline:** Coupling ingestion directly to indexing forced callers of the ingestion API to block until header extraction, regex tokenization, and thousands of B-tree insertions finished, preventing high-throughput batch ingestion.
4. **Absence of Crash-Recovery and Idempotency Guarantees:** Without durable message queues, atomic per-book claims, or transactional outbox markers, a process crash mid-indexing could leave partial state in the database, lose pending indexing notifications, or falsely mark a book as indexed before its postings were committed.

1.1 Stage 2 Objectives and Scope

The primary objective of **Stage 2** [1] is to re-architect the monolithic data pipeline into a cooperative **Service-Oriented Architecture (SOA)** deployed on a **single host** (*single-host multi-container deployment*) via Docker Compose, while establishing the architectural boundaries, formal API contracts, and coordination protocols needed for a multi-node distributed cluster in Stage 3. Specifically, Group **FourByTees** (`com.fourbytees`) achieved the following technical objectives:

- **Modular Service Decomposition:** Decomposed the codebase into a 4-module Maven reactor (`common-dto`, `ingestion-service`, `indexing-service`, and `search-service`) compiled under Java 17 (`maven.compiler.release=17`) with Javalin 6.1.3 [4] and Gson 2.11.0.
- **Formal REST & Non-HTTP Contracts:** Separated public **Front-End-Facing APIs** (served by `search-service` replicas behind Nginx on port `7000`) from **Internal & Operational APIs** (`ingestion-service` on port `7001` and `indexing-service` on internal port `7002`), and formally specified all HTTP endpoints, filesystem markers (`.done`, `.published`), JMS queues (`books.downloaded`, `books.indexed`, `books.dlq`), and Hazelcast distributed structures in `docs/openapi.yaml` [11].
- **Hybrid Event-Driven Choreography and Fault Tolerance:** Implemented at-least-once event delivery using a dual Transactional Outbox pattern [13] (`OutboxService` on disk for ingestion and `BookIndexedPublisher` in Hazelcast for indexing), lease-based per-book claim coordination (`BookClaim`) with automatic takeover after `INDEXING_TIMEOUT_MS`, exponential JMS redelivery backoff, and zero-downtime atomic full-index rebuilds (`IndexRebuilder`).
- **Methodologically Rigorous Benchmarking:** Isolated JMH 1.37 microbenchmarks [8] under `src/test/java/com/fourbytees/benchmark/` to keep production artifacts clean, and automated full-stack coordination, indexing, HTTP concurrency ($c \in \{1, 4, 8, 16, 32\}$), and container telemetry benchmarks via `scripts/run_benchmarks.ps1` [3].

2 System Architecture

The Stage 2 system is composed of six containerized services orchestrated on a single physical host by `docker-compose.yml`: three custom Java 17 microservices (`ingestion-service`, `indexing-service`, and `search-service`), an HTTP reverse proxy and load balancer (`nginx`), a persistent JMS message broker (`activemq`), and an in-memory distributed coordination server (`hazelcast`).

2.1 Architectural Topology and Communication Boundaries

Figure 1 presents the complete single-host SOA topology, explicitly distinguishing four architectural boundaries:

1. **Front-End / External HTTP Boundary (Host Port `7000`):** External web/mobile clients and benchmark runners interact exclusively with the `nginx` gateway on `http://localhost:7000`, which load-balances read-only requests (`GET /search`, `GET /books/{id}`, `GET /search/filters`, `GET /status`) across $R_s =$ `SEARCH_REPLICAS` (default 2) stateless `search-service` containers via round-robin upstream routing (`nginx/nginx.conf`).

2. **Internal & Operational HTTP Boundary (Port `7001` and Internal Port `7002`):** Seed ingestion scripts (`seed/download_seed.ps1`) and administrative controllers invoke `ingestion-service` on `http://localhost:7001` (`POST /ingest/{id}`, `GET /ingest/status/{id}`, `GET /ingest/list`, `GET /status`). Administrative indexing endpoints on `indexing-service` (`POST /index/update/{id}`, `POST /index/rebuild`, `GET /index/status`, `GET /status`) bind to port `7002` strictly inside the internal Docker bridge network and are never exposed directly to the public internet.
3. **Asynchronous Event Choreography & Hazelcast Instance-Client Topology:** `ingestion-service` and `indexing-service` are decoupled asynchronously via Apache ActiveMQ Classic (`tcp://activemq:61616`) using persistent JMS queues (`books.downloaded`, `books.indexed`, and `books.dlq`). Following the Hazelcast **Instance (Cluster Member) vs. Client** architectural model [5], the dedicated `hazelcast` container (`hazelcast:5701`) acts as the stateful cluster **Instance/Member** holding partitioned in-memory data structures and CP locks, while all microservice containers (`ingestion`, `indexing`, and `search`) connect as lightweight **Hazelcast Clients** (`HazelcastClient.newHazelcastClient()`) via `HazelcastClientFactory`.
4. **Shared Single-Host Storage Volumes:** Within the single-host Docker Compose boundary, `./datalake` is bind-mounted at `/data/datalake` into `ingestion-service` (read/write) and `indexing-service` (read-only consumption of book files), while `./datamarts` is bind-mounted at `/data/datamarts` into `indexing-service` (single-writer under `FencedLock("index-writer")`) and `search-service` (concurrent read-only connections in SQLite WAL mode).

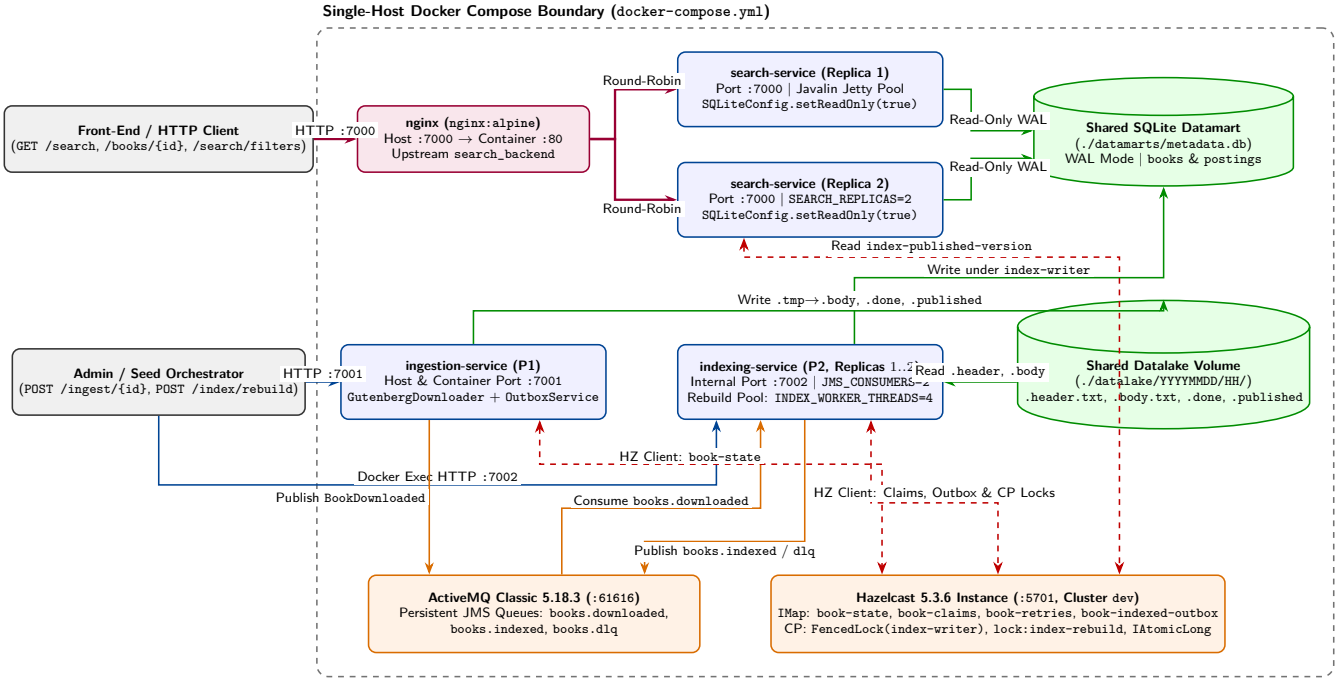


Figure 1: Stage 2 Service-Oriented Architecture (SOA) showing external Front-End HTTP routing, internal/operational HTTP endpoints, ActiveMQ JMS queues, Hazelcast Client-to-Instance coordination structures, and shared volumes within the single-host Docker Compose boundary.

2.2 Container Inventory, Resource Limits, and Concurrency Hierarchy

Table 1 details the six services configured in `docker-compose.yml` and `.env.example`, including their base images, port mappings, JVM heap flags (`-Xmx`), Docker cgroup CPU/RAM limits, and default replica counts.

3 API Design and Contracts

All synchronous HTTP interfaces and asynchronous non-HTTP coordination protocols are formally documented in `docs/openapi.yaml` (OpenAPI 3.0.3 [11]). In accordance with SOA design principles, we separate **Front-End-Facing APIs** from **Internal & Operational Service APIs**.

3.1 Front-End-Facing HTTP API (`search-service` via Nginx on Port `7000`)

Table 2 inventories the public read-only endpoints exposed to end-user applications through the Nginx reverse proxy (`http://localhost:7000`).

Table 1: Stage 2 Docker Compose service inventory, port mappings, JVM flags, and cgroup resource limits.

Service	Base Image / Runtime	Host	Int.	Replicas	CPU / RAM / Heap	Primary Responsibility
nginx	nginx:alpine	7000	80	1	0.5 CPU / 128M / N/A	Reverse proxy (worker_connections 1024) load-balancing search:7000.
search	eclipse-temurin:17-jre-jammy	None	7000	2 (SEARCH_REPLICAS)	1.0 CPU / 768M / -Xmx512m	Stateless read-only SQLite WAL query engine and Hazelcast client version reader.
ingestion	eclipse-temurin:17-jre-jammy	7001	7001	1	1.0 CPU / 768M / -Xmx512m	Gutenberg downloader, atomic datalake storage, and OutboxService publisher.
indexing	eclipse-temurin:17-jre-jammy	None	7002	1..2 (INDEXING_REPLICAS)	1.5 CPU / 1024M / -Xmx512m	Multi-consumer JMS indexer (JMS_CONSUMERS=2), IndexRebuilder (INDEX_WORKER_THREADS=4).
activemq	apache/activemq-classic:5.18.3	61616	61616	1	1.0 CPU / 512M / -Xmx256m	Persistent OpenWire JMS broker (KahaDB store in ./activemq-data).
hazelcast	hazelcast/hazelcast:5.3.6	5701	5701	1	1.0 CPU / 512M / Default	Stateful Hazelcast cluster **Instance** (hazelcast/hazelcast.yaml, cluster dev).

Table 2: Front-End-Facing HTTP API exposed by search-service behind Nginx (http://localhost:7000).

Method & Path	Query / Path Parameters	Status Codes	Contract, Ranking, Pagination, and Error Semantics
GET /search	q (req., string); author (opt., substring); language (opt., ISO-639-1); year (opt., 1000..9999); offset (opt., ≥ 0, def. 0); limit (opt., 1..100, def. 20)	200 OK, 400 Bad Req., 500 Int. Err.	Single-term full-text search. Extracts the first token matching \b[a-z]{3,}\b. Stop words or < 3-letter queries return 200 OK with total: 0, results: []. Ranked deterministically by p.freq DESC, b.book_id ASC. total reports the unpaginated global match count via SQL COUNT(*) OVER(), whereas results.size() is bounded by limit (≤ 100).
GET /books/{id}	id (req. path integer)	200 OK, 400 Bad Req., 404 Not Found, 500 Int. Err.	Returns book metadata JSON: {book_id, title, author, language, year}, where language and year are nullable. Non-integer id returns 400; absent book_id returns 404.
GET /search/filters	None	200 OK, 500 Int. Err.	Returns sorted distinct non-null filter domains currently indexed in SQLite: {"languages": ["en","es","fr"], "years": [1975,...,2008]}.

Strict Distinction Between 200 OK (Empty), 400 Bad Request, 404 Not Found, and 500 Internal Server Error:

- **200 OK with Empty Results:** Returned when q is syntactically valid (non-blank) and all filter parameters are valid, even if zero documents match, q is a stop word (e.g., q=the), or q contains no token of length ≥ 3 (e.g., q=hi).
- **400 Bad Request:** Returned with {"error": "Bad Request", "message": "..."} when q is missing or blank, offset < 0 or non-integer, limit < 1 or limit > 100 or non-integer, year $\notin [1000, 9999]$ or non-integer, language is not in the 12-code ISO-639-1 whitelist (en, es, fr, de, it, pt, la, nl, ru, zh, ja, el), or {id} in /books/{id} is non-numeric.
- **500 Internal Server Error:** Handled centrally by ApiErrors.internalServerError(ctx, gson), which logs the exception server-side via SLF4J and returns strictly {"error": "Internal server error"}, preventing any leakage of SQLite paths, table names, or stack traces.

3.2 Internal and Operational Service HTTP APIs

Table 3 catalogs the internal control-plane and operational endpoints exposed by ingestion-service (:7001), indexing-service (:7002), and search-service (:7000).

3.3 Non-HTTP Asynchronous Messaging and Distributed Coordination Contracts

Table 4 specifies all asynchronous queues in ActiveMQ Classic 5.18.3, filesystem state markers in ./datalake, and distributed coordination structures in Hazelcast 5.3.6 defined in SystemConstants.java and docs/openapi.yaml.

3.4 Functional, End-to-End, and Resilience Validation Suite

Contract compliance is verified through three complementary validation suites:

1. **Interactive HTTP Contract Suite (tests/search-api-tests.http):** Contains **22 HTTP test requests** covering health checks (#1), filter discovery (#2), single and combined filters (#3-#7), pagination (#8-#9), empty matches and stop words returning 200 OK (#10-#11), nine negative 400 Bad Request boundary checks (#12-#20: missing q, limit=-1, limit=200, limit=abc, year=abc, year=999, offset=-5, offset=xyz, language=klingon), and GET /books/{id} 200/404 checks (#21-#22).

Table 3: Internal and operational HTTP endpoints across `ingestion-service`, `indexing-service`, and `search-service`.

Service	Method & Path	Layer	Codes	Completion & Idempotency Contract
ingestion (:7001)	POST /ingest/{id}	Internal	200, 400, 404, 500	Synchronously downloads book id from Gutenberg, writes .header.txt, .body.txt, and .done, and triggers OutboxService.publishBook() (status: "downloaded"). If .done already exists, skips download and returns 200 OK ("already_downloaded"), re-running outbox publication if .published is missing. Returns 404 (FAILED_404) if absent on Gutenberg.
ingestion (:7001)	GET /ingest/status/{id}	Internal	200, 400	Checks whether <id>.done exists in ./datalake; returns {book_id, status: "available" | "not_found"}.
ingestion (:7001)	GET /ingest/list	Operational	200	Walks ./datalake (depth ≤ 4) for *.done files and returns {count, books: [id1, ...]}.
ingestion (:7001)	GET /status	Operational	200	Returns {"service": "ingestion-service", "status": "running"}.
indexing (:7002)	POST /index/update/{id}	Internal	200, 400, 404, 500	Synchronously parses book id from ./datalake and updates SQLite under FencedLock("index-writer") (force=true). Returns 404 if no .done marker exists.
indexing (:7002)	POST /index/rebuild	Operational	200, 409, 500	Builds replacement index in metadata.db.rebuild-<uuid>.tmp under lock:index-rebuild and swaps rows atomically under index-writer. Returns {books_processed, elapsed_time}. Concurrent calls fail fast with 409 Conflict.
indexing (:7002)	GET /index/status	Operational	200	Returns {books_indexed, last_update, index_size_MB} from SQLite books count and file byte length.
indexing (:7002)	GET /status	Operational	200	Returns {"service": "indexing-service", "status": "running"}.
search (:7000)	GET /status	Operational	200	Reports service, status ("running" or "degraded"), hazelcast_status ("connected" / "disconnected"), sqlite_status ("read-only-connected" / "error" via SELECT 1), and active_index_version.

Table 4: Non-HTTP event queues, filesystem markers, and Hazelcast distributed coordination contracts.

Subsystem	Identifier	Type / Primitive	Schema, Producer/Consumer, and Consistency Protocol
Datalake FS	<id>.done, <id>.published	Filesystem Markers	<id>.done ("READY", 5 B) is written after <id>.header.txt and <id>.tmp → <id>.body.txt (ATOMIC_MOVE). <id>.published ("PUBLISHED", 9 B) is written after ActiveMQ accepts BookDownloaded.
ActiveMQ 5.18.3	books.downloaded	JMS Queue (PERSISTENT)	Producer: OutboxService (AUTO_ACKNOWLEDGE). Consumer: BookDownloadedConsumer (CLIENT_ACKNOWLEDGE). Payload (BookDownloaded): {"event_id": "<uuid>", "book_id": 1342, "path": "...", "downloaded_at": "<ISO-8601>"}.
ActiveMQ 5.18.3	books.indexed	JMS Queue (PERSISTENT)	Producer: BookIndexedPublisher (backed by Hazelcast book-indexed-outbox). Payload (BookIndexed): {"event_id": "<uuid>", "book_id": 1342, "index_version": 25, "indexed_at": "<ISO-8601>"}.
ActiveMQ 5.18.3	books.dlq	Dead-Letter Queue	Producer: BookDownloadedConsumer when book-retries reaches MAX_RETRIES (default 3). Payload: {"book_id": 1342, "error": "...", "attempts": 3}.
Hazelcast AP	book-state	IMap<Integer, String>	Tracks book lifecycle: DOWNLOADED → INDEXING → INDEXED (or FAILED). State transitions during JMS consumption are guarded by per-key lock bookState.lock(bookId).
Hazelcast AP	book-claims	IMap<Integer, BookClaim>	Stores BookClaim(String claimantId, long claimedAtMs). Active claims younger than INDEXING_TIMEOUT_MS (300,000 ms) yield RETRY_LATER; expired claims are taken over.
Hazelcast AP	book-retries	IMap<Integer, Integer>	Tracks consecutive indexing exceptions per book_id. Removed upon INDEXED or ALREADY_INDEXED.
Hazelcast AP	book-indexed-outbox	IMap<String, String>	Keyed by event_id, stores JSON BookIndexed payloads recorded under LOCK_INDEX_WRITER until BookIndexedPublisher confirms JMS delivery.
Hazelcast CP	index-writer	FencedLock	Serializes all SQLite write transactions (saveBookIndex and replaceContentsFrom), index-published-version increments, and book-indexed-outbox recording.
Hazelcast CP	lock:index-rebuild	FencedLock	Acquired via non-blocking tryLock() by IndexRebuilder to guarantee at most one active rebuild across all replicas (409 Conflict on contention).
Hazelcast CP	index-published-version	IAtomicLong	Monotonically increasing index generation counter incremented on each book commit and full rebuild; exposed by search-service on GET /status.

2. **Automated End-to-End Validation Scripts (scripts/test_end_to_end.ps1 and .sh):** Executed via `powershell -NoProfile -ExecutionPolicy Bypass -File ".\scripts\test_end_to_end.ps1"`. The script verifies container readiness across ports 7000 and 7001 plus the running `indexing` container, invokes `seed/download_seed.ps1` to ingest all 25 seed books idempotently, polls `/search` until English (`q=adventure&language=en`), Spanish (`q=quijote&language=es`), and French (`q=candide&language=fr`) books are searchable, and executes **9 automated assertions** (**9/9 passed**).
3. **Automated Maven Unit & Integration Test Suite (`mvn clean test`):** Executes **41 JUnit 5 tests** across the reactor using embedded ActiveMQ (`vm://`) brokers, local Hazelcast instances, and isolated `@TempDir` SQLite databases (detailed in Section 5.5).

4 Microservice Implementation

The repository is organized as a multi-module Maven reactor (`pom.xml`, group `com.fourbytees`, artifact `stage_2`, version `1.0.0`) managing centralized dependency versions: Javalin `6.1.3`, Gson `2.11.0`, Hazelcast `5.3.6`, ActiveMQ `5.18.3`, Xerial SQLite JDBC `3.45.1.0`, SLF4J `2.0.9`, JUnit Jupiter `5.10.2`, and OpenJDK JMH `1.37`.

4.1 Code Hygiene and Benchmark Isolation (Addressing Stage 1 Feedback #41)

To maintain strict separation between production code and experimental harnesses:

- All JMH microbenchmark classes—`IndexingMicroBenchmark.java` (in `indexing-service`) and `SearchMicroBenchmark.java` (in `search-service`)—reside exclusively under `src/test/java/com/fourbytees/benchmark/`.
- In the root `pom.xml`, `jmh-core`, `jmh-generator-annprocess`, `junit-jupiter`, and `activemq-broker` are declared strictly with `<scope>test</scope>`. Consequently, the production shaded JARs built by `maven-shade-plugin` in the multi-stage Dockerfiles (`mvn -pl <service> -am clean package -DskipTests`) contain zero benchmark or test classes.
- Build-time `dependency-reduced-pom.xml` files are suppressed via `createDependencyReducedPom=false` in `pom.xml` and ignored in `.gitignore`, ensuring a clean working tree.

4.2 Shared Module (`common-dto`)

`common-dto` encapsulates cross-service constants and wire contracts:

- `SystemConstants`: Defines the canonical state names (`DOWNLOADED`, `INDEXING`, `INDEXED`, `FAILED`), Hazelcast map names (`book-state`, `book-claims`, `book-retries`), CP lock/atomic names (`index-writer`, `index-published-version`), and ActiveMQ queue names (`books.downloaded`, `books.indexed`, `books.dlq`).
- `BookDownloaded` and `BookIndexed`: Serializable DTOs with snake_case JSON fields (`event_id`, `book_id`, `path`, `downloaded_at` and `event_id`, `book_id`, `index_version`, `indexed_at`).
- `HazelcastClientFactory` and `JmsClientFactory`: Instantiate lightweight `HazelcastClient` connections (`HazelcastClient.newHazelcastClient(clientConfig)`) targeting the cluster member at `HAZELCAST_HOST:5701` with a 60 s connect timeout (`setClusterConnectTimeoutMillis(60000)`) and OpenWire JMS connections.

4.3 Ingestion Service (`ingestion-service`)

Implemented in package `com.fourbytees.ingestion` (`App`, `IngestionController`, `GutenbergDownloader`, `DatalakeStorage`, `OutboxService`):

1. **Gutenberg Acquisition & Header/Body Splitting (`GutenbergDownloader`):** Fetches `https://www.gutenberg.org/cache/epub/{id}/pg{id}.txt` using Java 11+ `HttpClient` (10 s connect timeout, 15 s request timeout, normal redirect following) with up to 3 attempts and linear backoff (1 s, 2 s). If Gutenberg returns HTTP 404, it throws `RuntimeException("FAILED_404")`, which `IngestionController` maps to HTTP 404. On HTTP 200, `splitHeaderAndBody()` locates the line containing `"*** START OF"`, extracting everything before it as `header` and everything after the marker line as `body`.
2. **Atomic Datalake Persistence (`DatalakeStorage`):** Partitions files by ingestion date and hour (`datalake/<yyyyMMdd>/<HH>/`). To prevent `indexing-service` from ever reading a half-written book body, `saveBook()` writes `<id>.header.txt`, writes the body to a temporary file `<id>.tmp`, atomically renames `<id>.tmp` → `<id>.body.txt` via `Files.move(..., REPLACE_EXISTING, ATOMIC_MOVE)`, and finally creates the completion marker `<id>.done` containing `"READY"`.
3. **Crash-Consistent Transactional Outbox (`OutboxService`):** Only after `<id>.done` is durable on disk does `OutboxService.publishBook(bookId, path)` execute the three-step publication protocol: (i) put `STATE_DOWNLOADED` into Hazelcast `IMap("book-state")`, (ii) send a `DeliveryMode.PERSISTENT` JSON `BookDownloaded` message to ActiveMQ queue `books.downloaded`, and (iii) create `<id>.published`

("PUBLISHED") on disk. At service startup (App.main), outboxService.runOutboxCheck() scans all *.done files in ./datalake and automatically republishes any book lacking a corresponding <id>.published marker.

4.4 Indexing Service (indexing-service)

Implemented in package com.fourbytees.indexing (App, IndexingConfig, IndexingController, BookDownloadedConsumer, JmsClientFactory, HeaderParser, BodyTokenizer, SqliteRepository, BookClaim, BookIndexedPublisher, IndexingOrchestrator, IndexRebuilder):

1. **Header Metadata Extraction (HeaderParser):** Streams <id>.header.txt line-by-line in UTF-8, supporting multiline indented continuation lines for Title: and Author:, extracting the 4-digit release/posting year via \b(1\d{3}|20\d{2})\b from Release date: or Posting date: lines, and normalizing the primary Language: name into a 12-language ISO-639-1 code (english→en, spanish→es, french→fr, etc., or null if unrecognized).
2. **Frozen Stage 1 Tokenization Contract (BodyTokenizer):** Converts <id>.body.txt to lowercase and extracts tokens matching \b[a-z]{3,}\b. Every regex match increments the 0-based wordIndex counter *before* checking the 30-word STOP_WORDS set (a, an, and, are, as, at, be, but, by, for, if, in, into, is, it, no, not, of, on, or, such, that, the, their, then, there, these, they, this, to), ensuring positional postings Map<String, List<Integer>> preserve exact word positions without renumbering.
3. **Lease-Based Claim Coordination (IndexingOrchestrator & BookClaim):** When BookDownloadedConsumer receives a message from books.downloaded, claimBook(bookId, bookState, bookClaims) acquires the key-level Hazelcast lock bookState.lock(bookId) and evaluates the state machine:
   - If currentState == INDEXED, returns IndexingResult.ALREADY_INDEXED (duplicate message acknowledged immediately).
   - If currentState == DOWNLOADED, transitions book-state to INDEXING, writes new BookClaim(claimantId, now) to book-claims, and proceeds to parse and index.
   - If currentState == INDEXING, inspects the existing BookClaim: if now - claimedAtMs < INDEXING_TIMEOUT_MS (300,000 ms), returns RETRY_LATER; if the claim has expired (≥ 300,000 ms), takes over the stale claim with a fresh timestamp and proceeds.
4. **Transactional SQLite Persistence & Post-Commit Outbox (SqliteRepository & BookIndexedPublisher):** Parsing (parseBook) runs *outside* the global write lock so multiple threads/replicas tokenize in parallel. Once parsed, processBook() acquires FencedLock("index-writer"):

```
FencedLock lock = hazelcast.getCPSubsystem().getLock(SystemConstants.LOCK_INDEX_WRITER);
lock.lock();
try {
    repository.saveBookIndex(metadata, parsed.postings()); // Single SQLite transaction (DELETE + INSERT)
    newVersion = hazelcast.getCPSubsystem()
            .getAtomicLong(SystemConstants.ATOMIC_INDEX_VERSION)
            .incrementAndGet();
    event = new BookIndexed(UUID.randomUUID().toString(), bookId, newVersion, Instant.now().toString());
    publisher.record(event); // Recorded in Hazelcast IMap("book-indexed-outbox") before unlock
    bookState.put(bookId, SystemConstants.STATE_INDEXED);
    if (bookClaims != null) {
        removeOwnClaim(bookId, bookClaims);
    }
} finally {
    lock.unlock();
}
publisher.publish(event); // Bounded post-commit retries + background outbox relay
```

Listing 1: Coordinated SQLite commit and post-commit outbox recording in IndexingOrchestrator.java.

5. **Zero-Downtime Atomic Full Rebuild (IndexRebuilder):** Guarded by an in-process AtomicBoolean and distributed FencedLock("lock:index-rebuild") (tryLock()). Builds a complete replacement database at metadata.db.rebuild-<uuid>.tmp using INDEX_WORKER_THREADS (4) parallel parser threads in batches of 2 × workerThreads = 8. Once built, it acquires FencedLock("index-writer"), indexes any late-arriving books that became ready during the build, and calls repository.replaceContentsFrom(temporaryDb), which executes ATTACH DATABASE ? AS replacement and swaps main.books and main.postings inside a single BEGIN IMMEDIATE ... COMMIT transaction followed by PRAGMA wal_checkpoint(PASSIVE) and cleanup of .tmp, -wal, and -shm side files.

4.5 Search Service (search-service)

Implemented in package com.fourbytees.search (Main, HazelcastConfig, SearchController, BookController, StatusController, ApiErrors, DatabaseManager):

- **Read-Only WAL Query Execution (DatabaseManager):** Opens every JDBC connection with SQLiteConfig.setReadOnly(true). searchByTerm() normalizes q with the frozen Stage 1 regex \b[a-z]{3,}\b and 30-word stop-word filter, then executes a single parameterized query joining postings p and books b with window function COUNT(*) OVER() AS total_matches, ordered by p.freq DESC, b.book_id ASC with LIMIT

`? OFFSET ?`. Because $\text{idf}(t) = \ln(N/\text{df}_t)$ is constant across all documents matching a single query term t, ordering by $\text{freq}(t, d)$ descending yields the exact relative single-term TF-IDF ranking.

- **Graceful Degradation (`HazelcastConfig` & `StatusController`):** If Hazelcast is unreachable, `search-service` continues serving SQLite queries while reporting `"status": "degraded"`, `"hazelcast_status": "disconnected"`, and `"active_index_version": "unknown"` on `GET /status`.

5 Design Decisions

5.1 Hybrid Coordination (ActiveMQ + Hazelcast + Dual Outbox) vs. Centralized Orchestration

A purely centralized orchestrator (where a single controller service synchronously calls ingestion, waits for completion, calls indexing, and updates search replicas via HTTP RPCs) introduces tight temporal coupling and a single point of orchestration failure [13]. Instead, our hybrid architecture separates **asynchronous work distribution** (ActiveMQ JMS queues) from **shared state & mutual exclusion** (Hazelcast CP/AP primitives):

- **Temporal Decoupling via Competing Consumers:** `ingestion-service` returns `HTTP 200` as soon as the book is sealed in `./datalake` with `<id>.done` and queued in `books.downloaded`. If `indexing-service` is temporarily busy with a large novel (e.g., *Moby Dick*), ingestion continues unblocked while ActiveMQ buffers pending events on disk (KahaDB).
- **Elimination of Dual-Write Anomalies via Dual Outbox:** Writing to a filesystem/database and publishing to a message broker is a classic non-atomic dual write [14]. We solve this in both services without heavyweight XA two-phase commit:
  1. In `ingestion-service`, `<id>.done` acts as the durable commit point and `<id>.published` as the outbox completion marker scanned by `OutboxService.runOutboxCheck()`.
  2. In `indexing-service`, `BookIndexedPublisher.record(event)` writes the serialized `BookIndexed` event into Hazelcast `IMap("book-indexed-outbox")` *while still holding* `FencedLock("index-writer")`. If the broker fails after the SQLite commit, the background thread (`book-indexed-outbox-relay`, running every `OUTBOX_FLUSH_INTERVAL_MS = 10,000 ms` for entries older than `OUTBOX_MIN_AGE_MS = 5,000 ms`) delivers the event later **without ever repeating the SQLite write**.

5.2 Three-Tier Concurrency Model: Clients, Container Replicas, and Process Threads

We explicitly distinguish three orthogonal concurrency dimensions in our architecture:

1. **HTTP Client Concurrency ($c \in \{1, 4, 8, 16, 32\}$):** Number of simultaneous external HTTP requests arriving at Nginx (`:7000`). Nginx multiplexes connections asynchronously (`worker_connections 1024`) across upstream `search-service` containers.
2. **Container / Process Replicas (`SEARCH_REPLICAS=2`, `INDEXING_REPLICAS=1..2`):** Separate OS containers with isolated JVM heaps (`-Xmx512m`). Scaling `SEARCH_REPLICAS` increases parallel read capacity across CPU cores; scaling `INDEXING_REPLICAS` increases parallel document parsing and JMS consumption across processes.
3. **Intra-Process Threads:**
   - *Javalin/Jetty HTTP Threads:* Handle incoming REST requests inside each microservice.
   - *JMS Consumer Threads (`JMS_CONSUMERS=2`):* Dedicated daemon threads (`jms-consumer-0`, `jms-consumer-1`) inside each `indexing-service` replica pulling messages concurrently from `books.downloaded`.
   - *Rebuild Worker Threads (`INDEX_WORKER_THREADS=4`):* Fixed thread pool (`index-rebuild-1..4`) inside `IndexRebuilder` that parses and tokenizes batches of $2 \times 4 = 8$ books in parallel during `POST /index/rebuild`.

5.3 Shared State Protection: Datalake Atomicity and SQLite WAL Locking

- **Preventing Conflicting or Partial Reads in the Datalake:** `DatalakeStorage.saveBook()` is `synchronized` within `ingestion-service`, writes the body to `<id>.tmp`, performs an atomic filesystem rename (`StandardCopyOption.ATOMIC_MOVE`) to `<id>.body.txt`, and writes `<id>.done` last. `IndexingOrchestrator.findBookDirectory()` only resolves directories containing `<id>.done`, guaranteeing `indexing-service` never reads an incomplete download.
- **Preventing `SQLITE_BUSY` Between Writers and Readers:** SQLite is initialized with `PRAGMA journal_mode=WAL;` and `PRAGMA synchronous=NORMAL;`. In WAL mode [7], readers do not block writers and writers do not block readers. To prevent write-write conflicts (`SQLITE_BUSY`) across multiple `JMS_CONSUMERS` threads or multiple `indexing-service` container replicas, all SQLite write transactions (`saveBookIndex` and `replaceContentsFrom`) are serialized globally via Hazelcast CP `FencedLock("index-writer")`, while `replaceContentsFrom` additionally configures `PRAGMA busy_timeout=30000;`.

5.4 Comprehensive Quantitative Justification of Datalake, SQLite, and Hazelcast (Feedback #4, #10, #25, #27, #30)

Rather than justifying storage and coordination choices solely by query latency, Table 5 evaluates the complete operational trade-off across all five lifecycle dimensions using empirical data from our 25-book corpus (1,776,589 non-stop-word tokens, 192,167 postings across 66,727 distinct terms).

Table 5: Quantitative trade-off analysis of the Datalake, SQLite WAL Datamart, and Hazelcast coordination layer.

Lifecycle Dimension	Partitioned Datalake (`./datalake/YYYYMMDD/HH/`)	SQLite WAL Datamart (`./datamarts/metadata.db`)	Hazelcast 5.3.6 Coordination (`IMap` + CP Subsystem)
1. Initial Build / Full Rebuild Cost	**25 books ingested:** 100 files created across 2 directories (`20261101/20`). Sequential WAN download + atomic split completes in ∼ 18–25 s depending on Gutenberg bandwidth.	**Full 25-book atomic rebuild (`IndexRebuilder`):** 455.30 s (18.21 s/book avg.) on Docker WSL2 `9p` bind mount (192,167 postings + B-tree index `idx_postings_term`), with 0 ms reader downtime.	**Negligible rebuild overhead:** 1 `tryLock()` on `lock:index-rebuild`, 1 `lock()` on `index-writer` during swap, 1 `IAtomicLong` increment, and 25 `book-state` puts (∼ 2.6 ms total).
2. Incremental Per-Book Update Cost	$O(1)$ **file write:** Writes 4 files (`.header.txt`, `.tmp`→`.body.txt`, `.done`, `.published`) without touching existing books.	**Transactional `DELETE`+`INSERT`:** 10.48 ms (±6.99 ms) on native Windows NVMe SSD (JMH); 11.22–65.78 s over WSL2 `9p` bind mount due to cross-VM `fsync` serialization.	**Sub-millisecond coordination:** 102.78 μs/op (±18.42 μs in JMH) for claim put, state transition, `FencedLock` acquire/release, version increment, and claim removal.
3. Concurrent Read Latency & Throughput	Not queried on the user search path; scanned only during `GET /ingest/list` or `POST /index/rebuild`.	**Fast indexed B-tree reads:** 3.05 ms (`getBookById`), 4.27 ms (unfiltered `/search`), 4.53 ms (filtered `/search`) in JMH; 18.09 ms mean HTTP latency at $c = 8$ and 462.35 req/s peak throughput at $c = 32$.	**Off the critical search path:** `search-service` queries SQLite directly; Hazelcast `IAtomicLong` is read only on `GET /status`.
4. Disk & Memory Footprint	**100 files, 2 directories:** 16,325,932 B (15.57 MiB) logical size (20,943 B headers, 16,304,639 B bodies, 350 B markers); 16,666,624 B (15.89 MiB) allocated on NTFS (4 KB clusters).	**1 file (`metadata.db`):** 5,785 pages ×4,096 B = 23,695,360 B (22.60 MiB logical & allocated; 22.56 MiB right after initial 25-book build). Stores JSON `positions` per posting.	**In-memory container RSS:** 182.3 MiB (`stage_2-hazelcast-1`, 35.60% of 512 MiB limit) holding all maps and CP state for 25 books.
5. Consistency & Failure-Recovery Role	Immutable source of truth allowing complete index reconstruction (`POST /index/rebuild`) if `metadata.db` is lost or corrupted.	ACID transactions ensure a book's metadata and all its postings appear atomically or roll back completely on error.	Prevents duplicate concurrent indexing (`book-claims`), serializes single-writer SQLite access (`index-writer`), and buffers post-commit `BookIndexed` events.

5.5 Verification of Idempotency, Crash Recovery, and Failure Consistency (Feedback #9, #32, #33, #40)

To guarantee that partial failures never corrupt the index or falsely mark an unindexed book as `INDEXED` (directly resolving Stage 1 Feedback #40), we implemented and verified six failure-recovery invariants across **41 automated JUnit tests** (Table 6) and live container fault-injection tests:

How Stage 1 Feedback #40 (Never Marking Failed Indexing as Indexed) Is Strictly Prevented: In `IndexingOrchestrator.indexDownloadedBook()` and `processBook()`, a book's state in Hazelcast (`book-state`) is set to `INDEXED` *only inside the* `try` *block after* `parseBook(bookId)` and `repository.saveBookIndex(metadata, parsed.postings())` have both succeeded without throwing an exception. If `HeaderParser` or `BodyTokenizer` throws an `IOException`, execution never reaches `lock.lock()` or `saveBookIndex()`; the `catch` block in `indexDownloadedBook()` invokes `removeOwnClaim(bookId, bookClaims)`, and `BookDownloadedConsumer.handleFailure()` increments `book-retries` and either resets `book-state` to `DOWNLOADED` followed by `session.recover()` (triggering JMS redelivery) or, upon reaching `MAX_RETRIES`, sets `book-state` to `FAILED` and forwards the failure to `books.dlq`. Similarly, if `SqliteRepository.saveBookIndex()` encounters a `SQLException` mid-batch, `conn.rollback()` undoes all partial `DELETE`/`INSERT` statements before rethrowing the exception.

6 Experimental Methodology and Benchmarks

In strict compliance with empirical software engineering standards, this section first establishes our complete experimental methodology—hardware/software environment, exact 25-book dataset and query ground truth, mathematical timing boundaries, warm-up/repetition protocol, and artifact traceability (Section 6.1)—before presenting any component-level JMH microbenchmarks (Section 6.2) or system-level integration results (Section 6.3) [3].

Table 6: Summary of automated unit, integration, and fault-recovery test suites (`mvn clean test`, 41/41 passing).

Module	Test Class	Tests	Verified Consistency, Idempotency, and Failure-Recovery Scenarios
ingestion	IngestionServiceTest	2	(1) Idempotent saveBook() returning identical path when <id>.done exists; (2) Simulated crash after writing .done but before .published: verifies OutboxService.runOutboxCheck() detects and republishes the book exactly once to books.downloaded, transitions Hazelcast book-state to DOWNLOADED, writes .published, and emits zero duplicates on a second scan.
indexing	HeaderParserTest	6	Multiline indented Title/Author continuations, primary language extraction from multi-language headers, 12-code ISO-639-1 normalization, and safe null assignment for missing/unrecognized language or year.
indexing	IndexingServiceTest	5	(1) Idempotent re-indexing (DELETE+INSERT yields exactly 1 book row and exact freq=2 without duplication); (2) 2 concurrent workers racing on bookId=99: exactly 1 succeeds and 1 is rejected by BookClaim; (3) Recent claim (< INDEXING_TIMEOUT_MS) returns RETRY_LATER; (4) Expired claim ($\geq$ 100 ms timeout) is taken over, indexed, and cleared; (5) Duplicate delivery in state INDEXED returns ALREADY_INDEXED.
indexing	BookIndexedPublicationTest	4	(1) Broker down after SQLite commit: book stays INDEXED (CountingRepository.writes == 1), event stays in book-indexed-outbox, and flushPendingEvents() delivers it upon broker recovery **without repeating the SQLite write**; (2) Transient broker failure recovered within 3 immediate attempts; (3) Bounded retry exhaustion; (4) Lost ACK retry preserves identical event_id and index_version.
indexing	JmsRedeliveryTest	2	(1) End-to-end ActiveMQ redelivery after session.recover() on RETRY_LATER: consumer waits for initial claim expiration (350 ms) + exponential backoff and indexes the book exactly once; (2) Missing datalake files (NoSuchFileException) exhaust MAX_RETRIES=2, set book-state=FAILED, and route JSON error payload to books.dlq.
indexing	IndexRebuilderTest	5	(1) Complete replacement index built in .rebuild-<uuid>.tmp and swapped atomically, purging stale rows; (2) Simulated parse failure on book 5 rolls back rebuild, keeps previous index active, releases lock:index-rebuild, and deletes .tmp/-wal/-shm files; (3) Continuous concurrent read-only search thread during 60-book rebuild observes *only* old count (10) or new count (60), never 0 or partial counts; (4) Concurrent rebuild calls across replicas throw RebuildInProgressException; (5) Side-file cleanup verification.
indexing	IndexingControllerTest	3	Maps POST /index/rebuild outcomes to 200 OK, 409 Conflict (RebuildInProgressException), and generic 500 Internal Server Error (RebuildFailedException).
search	SearchServiceTest	14	TF-IDF ranking (freq DESC) and deterministic tie-breaking (book_id ASC), individual/combined filters (author, language, year), pagination, stop words returning 200 OK (total=0), 13 invalid parameter cases returning 400, degraded status when Hazelcast is down, and strict 500 sanitization (corrupt.db and IllegalStateException never leak SQL or paths).

6.1 Experimental Setup and Methodology

6.1.1 Hardware, Operating System, JVM, and Container Environment

All experiments were executed on a single physical machine whose specifications were captured automatically in `docs/benchmarks/benchmark-metadata.json` by `scripts/run_benchmarks.ps1` at `2026-11-01T21:07:49Z` on Git branch `feat/coordination-benchmarks` (commit `5c4ee06`). Table 7 details the exact hardware and software stack.

Table 7: Exact hardware, storage, operating system, JVM, and container runtime environment (`benchmark-metadata.json`).

Environment Dimension	Exact Measured Specification
Host Model	HP OMEN Transcend Gaming Laptop 14-fb1xxx
Processor (CPU)	Intel(R) Core(TM) Ultra 9 285H (16 cores, 16 logical processors, 2.90 GHz base clock)
Physical & VM RAM	64 GB physical LPDDR5x RAM (68, 073, 553, 920 bytes); 31.05 GiB allocated to the WSL2 Linux utility VM
Storage & Filesystem	Internal PCIe NVMe SSD formatted with Windows NTFS (4, 096-byte allocation unit / cluster size). Inside Docker Desktop containers, ./datalake and ./datamarts are bind-mounted from NTFS into the Linux container namespace via the WSL2 Plan 9 (9p/drvfs) virtual filesystem protocol.
Host & Guest OS	Host: Microsoft Windows 11 Pro (10.0.26300, Build 26300); Guest Container Kernel: Linux 6.18.54.1-microsoft-standard-WSL2
Host JDK (JMH)	Microsoft OpenJDK 17.0.20.1+1-LTS (OpenJDK 64-Bit Server VM, build 17.0.20.1+1-LTS); JMH fork flags: -Xms2g -Xmx2g
Container JRE & Flags	eclipse-temurin:17-jre-jammy; JAVA_OPTS="-Dfile.encoding=UTF-8" with -Xmx512m (ingestion, indexing, search) and -Xms128m -Xmx256m (activemq)
Build & Test Tooling	Apache Maven 3.9.6, OpenJDK JMH 1.37, JUnit Jupiter 5.10.2, Xerial sqlite-jdbc 3.45.1.0
Container Engine	Docker Desktop / Docker Engine 29.9.2 (build a82e528), Docker Compose v5.6.1

6.1.2 Exact Input Dataset: The 25-Book Multilingual Seed Corpus

The input dataset defined in `seed/book_ids.txt` consists of **25 complete Project Gutenberg books** (23 English, 1 Spanish, 1 French). Table 8 lists every book ID alongside its extracted metadata, exact byte sizes for `<id>.header.txt` and `<id>.body.txt` in `./datalake/20261101/20/`, distinct non-stop-word vocabulary terms (`COUNT(term)` in SQLite `postings`), and total non-stop-word token occurrences (`SUM(freq)`).

6.1.3 Exact Queries Executed and Ground-Truth Verification (Feedback #28, #33)

Table 9 documents the exact HTTP queries executed by `scripts/test_end_to_end.ps1` (9 validation assertions) and `scripts/run_benchmarks.ps1` (8-query workload mix), verifying that the expected and observed HTTP status

Table 8: Complete 25-book input dataset (`seed/book_ids.txt`) with exact file sizes, vocabulary cardinality, and token counts.

ID	Extracted Title	Extracted Author	Lang	Year	Hdr (B)	Body (B)	Body (KB)	Terms	Tokens
5	*The United States Constitution*	United States	en	1975	780	47,453	46.3	1,379	4,758
11	*Alice's Adventures in Wonderland*	Lewis Carroll	en	2008	824	173,394	169.3	2,907	19,010
16	*Peter Pan [Peter and Wendy]*	J. M. Barrie	en	2008	790	287,903	281.2	5,090	32,374
17	*The Book of Mormon*	Jr. Joseph Smith	en	2008	929	1,517,904	1,482.3	5,849	159,306
74	*The Adventures of Tom Sawyer*	Mark Twain	en	2004	811	433,441	423.3	7,491	48,283
76	*Adventures of Huckleberry Finn*	Mark Twain	en	2004	825	621,538	607.0	6,317	69,327
84	*Frankenstein; or, The Modern Prometheus*	Mary W. Shelley	en	1993	964	447,821	437.3	7,153	47,617
98	*A Tale of Two Cities*	Charles Dickens	en	1994	804	806,310	787.4	9,835	88,500
158	*Emma*	Jane Austen	en	1994	776	932,862	911.0	7,239	105,375
219	*Heart of Darkness*	Joseph Conrad	en	2006	797	236,193	230.7	5,674	26,032
345	*Dracula*	Bram Stoker	en	1995	805	889,437	868.6	9,324	99,285
844	*The Importance of Being Earnest*	Oscar Wilde	en	1997	879	140,828	137.5	2,904	15,500
1080	*A Modest Proposal*	Jonathan Swift	en	1997	813	38,754	37.8	1,507	3,991
1260	*Jane Eyre: An Autobiography*	Charlotte Brontë	en	1998	837	1,083,878	1,058.5	12,573	119,922
1342	*Pride and Prejudice*	Jane Austen	en	1998	921	771,351	753.3	6,864	83,936
1400	*Great Expectations*	Charles Dickens	en	1998	812	1,057,313	1,032.5	10,855	116,315
1661	*The Adventures of Sherlock Holmes*	Arthur Conan Doyle	en	1999	864	606,642	592.4	7,928	66,524
1952	*The Yellow Wallpaper*	C. Perkins Gilman	en	1999	830	51,654	50.4	1,644	5,422
2000	*Don Quijote*	M. de Cervantes	es	1999	840	2,224,885	2,172.7	17,818	227,469
2542	*A Doll's House : a play*	Henrik Ibsen	en	2001	848	168,142	164.2	2,738	18,977
2591	*Grimms' Fairy Tales*	J. & W. Grimm	en	2001	846	559,234	546.1	5,047	65,225
2701	*Moby Dick; Or, The Whale*	Herman Melville	en	2001	824	1,275,400	1,245.5	16,941	140,870
4300	*Ulysses*	James Joyce	en	2003	775	1,585,501	1,548.3	28,438	178,514
4650	*Candide, ou l'optimisme*	Voltaire	fr	2003	793	233,237	227.8	4,444	22,856
14925	*Punch, or the London Charivari, Vol. 1*	Various	en	2005	956	113,564	110.9	4,208	11,201
Corpus Totals ($N = 25$ books; 66,727 global unique terms across corpus)					**20,943**	**16,304,639**	**15,922.5**	**192,167**	**1,776,589**

codes, `total` counts, and ordered `book_id` lists match 100%.

Table 9: Exact query workload and ground-truth verification against the 25-book SQLite datamart (`metadata.db`).

Suite	Exact HTTP Request Path	Exp.	Obs.	total (sz)	Returned book_ids in Exact Order (freq DESC, book_id ASC)
E2E + Load #1	`GET /search?q=adventure`	200	200	13 (13)	1661 (f=19), 16 (f=14), 74 (f=14), 345 (f=9), 2701 (f=5), 76 (f=3), 1260 (f=3), 158 (f=2), 1342 (f=2), 1400 (f=2), 84 (f=1), 219 (f=1), 2591 (f=1).
E2E + Load #2	`GET /search?q=adventure&author=twain`	200	200	2 (2)	74 (*Tom Sawyer*, f=14), 76 (*Huckleberry Finn*, f=3).
E2E + Load #3	`GET /search?q=adventure&language=en`	200	200	13 (13)	Identical to `GET /search?q=adventure` (all 13 matching books have `language="en"`).
E2E + Load #4	`GET /search?q=quijote&language=es`	200	200	1 (1)	2000 (*Don Quijote*, Miguel de Cervantes Saavedra, `es`, `year=1999`, f=2242).
E2E + Load #5	`GET /search?q=candide&language=fr`	200	200	1 (1)	4650 (*Candide, ou l'optimisme*, Voltaire, `fr`, `year=2003`, f=405).
E2E + Load #6	`GET /search?q=adventure&limit=5&offset=0`	200	200	13 (5)	First 5 results: 1661 (f=19), 16 (f=14), 74 (f=14), 345 (f=9), 2701 (f=5).
E2E + Load #7	`GET /search?q=the` (stop word)	200	200	0 (0)	`results: []` (filtered by frozen 30-word `STOP_WORDS` contract).
Load #8	`GET /books/1342`	200	200	N/A (1)	`{book_id: 1342, title: "Pride and Prejudice", author: "Jane Austen", language: "en", year: 1998}`.
E2E #8	`GET /search` (missing `q`)	400	400	N/A	`{error: "Bad Request", message: "Parameter 'q' is mandatory."}`
E2E #9	`GET /search?q=adventure&limit=-1`	400	400	N/A	`{error: "Bad Request", message: "Parameter 'limit' must be between 1 and 100."}`

6.1.4 Mathematical Timing Boundaries (Feedback #2, #6, #14, #22)

To avoid ambiguity regarding what each latency metric includes or excludes, we define the exact start (t_{start}) and stop (t_{end}) boundaries of the high-resolution timer (`System.nanoTime()` in JMH and `System.Diagnostics.Stopwatch` in PowerShell):

1. **Ingestion Boundary Separation (WAN Download vs. Local Datalake Persistence vs. Outbox):** An HTTP `POST /ingest/{id}` request spans three distinct phases:

$$T_{\text{ingest_http}} = \underbrace{T_{\text{wan_download}}}_{\text{Gutenberg HTTP GET + split}} + \underbrace{T_{\text{datalake_save}}}_{\texttt{.header + .tmp}\rightarrow\texttt{.body + .done}} + \underbrace{T_{\text{outbox_publish}}}_{\text{Hazelcast put + ActiveMQ send + } \texttt{.published}}.$$

   Because $T_{\text{wan_download}}$ depends on external internet routing to `www.gutenberg.org` (100–900 ms jitter), we exclude WAN downloads from all indexing and search benchmarks by pre-populating `./datalake` via `seed/download_seed.ps1` prior to running `scripts/run_benchmarks.ps1`.
2. **Component Indexing Microbenchmarks (`IndexingMicroBenchmark.java`):** Measured in-process on the Windows host NVMe SSD (`@TempDir`):
   - $T_{\text{headerParsing}}$: Starts immediately before `HeaderParser.parse(1342, headerFile)` (921 B UTF-8 file read + regex extraction) and ends when the `BookMetadata` record is returned to JMH.
   - $T_{\text{bodyTokenization}}$: Starts immediately before `BodyTokenizer.tokenize(bodyFile)` ($\sim$ 44 KB synthetic 80-paragraph literary text read + lowercase + regex `\b[a-z]{3,}\b` + stop-word filtering) and ends when the `TokenizedResult` map is returned.
   - $T_{\text{hazelcastCoordinationCost}}$: Starts before `bookClaims.put(1342, claim)` and includes `bookState.put(INDEXING)`, `writerLock.lock()`, `bookState.put(INDEXED)`, `publishedVersion.incrementAndGet()`, `writerLock.unlock()`, and `bookClaims.remove(1342, claim)` on an embedded Hazelcast instance.
   - $T_{\text{sqliteIndexInsertion}}$: Starts before `sqliteRepository.saveBookIndex(parsedMetadata, parsedPostings)` (excluding parsing) and ends after `conn.commit()` and `getIndexedBooksCount()` complete on the local NVMe `benchmark_index.db`.
3. **Live Containerized Indexing Latency (`indexing-book-latency.csv`):** Measured via `Stopwatch.StartNew()` wrapping `docker compose exec -T indexing curl -s -X POST "http://localhost:7002/index/update/$id"`. This boundary excludes JMS queue wait time ($T_{\text{jms_queue_wait}} = 0$) and measures the complete synchronous container execution:

$$T_{\text{index_update}} = T_{\text{docker_exec}} + T_{\text{datalake_scan}} + T_{\text{parseBook}} + T_{\text{lock_wait}} + T_{\text{sqlite_wsl2_tx}} + T_{\text{hz_version_outbox}} + T_{\text{jms_publish}}.$$

4. **Search Microbenchmarks vs. Live Nginx Concurrent HTTP Search:**
   - $T_{\text{sqliteGetBookById}}$, $T_{\text{sqliteSearchUnfiltered}}$, $T_{\text{sqliteSearchFiltered}}$ (`SearchMicroBenchmark`): Start at the Java method invocation on `DatabaseManager` and end when the `Book` or `SearchResult` object is returned

(pure SQLite read-only JDBC query over a 25-book, 5,000-posting database on native NVMe, with zero HTTP/JSON overhead).

- $T_{\text{httpSearchClientQuery}}$ (`SearchMicroBenchmark`): Starts at `httpClient.send(searchHttpRequest, ...)` and ends when the HTTP `200` body is received from an in-process Javalin server (`localhost:<ephemeralPort>`), adding loopback TCP + Jetty + Gson serialization to DAO execution.
- $T_{\text{http_concurrency}}$ (`http-search-concurrency.csv`): For each request i inside worker runspace $w \in \{1..c\}$, `Stopwatch` starts immediately before `$client.GetAsync("$BaseUrl$q")` and stops immediately after the HTTP response headers/status arrive from `http://localhost:7000` (traversing Windows TCP loopback → Docker port proxy → `stage_2-nginx-1` → Docker bridge → `stage_2-search-1/2` → WSL2 `9p` read-only SQLite WAL query → JSON response). Throughput is computed over the outer wall-clock barrier T_{wall} from launching all c concurrent workers until the last worker completes its 25th request: Throughput $= (25 \times c)/(T_{\text{wall}}/1000)$.

6.1.5 Preparation, Cache State, Warm-up, and Statistical Protocol (Feedback #37, #38)

- **Pre-Benchmark State Preparation:** Before running `scripts/run_benchmarks.ps1`, the Docker Compose stack is started with `SEARCH_REPLICAS=2` and `INDEXING_REPLICAS=2`, all 25 seed books are ingested and indexed (`books_indexed: 25`), and ActiveMQ queues are drained (`0` pending messages).
- **JMH Warm-up & Measurement Protocol:** Both JMH suites execute inside a dedicated forked JVM (`forks = 1`, `-Xms2g -Xmx2g`) with **5 warm-up iterations** (400 ms continuous execution per iteration) to stabilize HotSpot C2 JIT compilation and OS page caches, followed by **10 measurement iterations** (400 ms each). From the 10 iteration means in `rawData`, we report the sample mean, the 99.9% Student-t confidence interval half-width (`Score Error (99.9%)`), the sample standard deviation (StdDev), the minimum (p_0), median (p_{50}), and 95th percentile (p_{95}).
- **HTTP Load Warm-up & Statistical Aggregation:** Before starting the concurrency sweep, `scripts/run_benchmarks.ps1` executes **16 warm-up HTTP requests** (2 full passes over the 8-query mix) against `http://localhost:7000` so that Jetty thread pools, SQLite WAL page caches, and Nginx upstream connections are warm. For each concurrency level $c \in \{1, 4, 8, 16, 32\}$, c parallel PowerShell runspaces each execute 25 sequential requests using a persistent `System.Net.Http.HttpClient` ($N = 25c$ samples per level: $25, 100, 200, 400, 800$). Across all N individual request latencies, the script computes the exact arithmetic mean, unbiased sample standard deviation $s = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(x_i - \bar{x})^2}$, median ($p_{50}$), p_{95}, and p_{99}.

6.1.6 Traceability and Reproducibility Matrix (Feedback #11, #39)

Table 10 links every experimental result table in this report to its exact execution command, Git commit (`5c4ee06`), and tracked raw data files in `docs/benchmarks/`.

Table 10: End-to-end traceability matrix mapping report tables to execution commands and raw data artifacts.

Report Table	Commit	Tracked Artifact Files (docs/benchmarks/)	Exact Reproduction Command
Table 7 (Environment)	5c4ee06	benchmark-metadata.json	powershell -ExecutionPolicy Bypass -File .\scripts\run_benchmarks.ps1 -ResultsDir docs/benchmarks (Step 0)
Table 11 (JMH Indexing)	5c4ee06	jmh-indexing.csv, jmh-indexing.json	java -cp <indexing-test-cp> com.fourbytees.benchmark.IndexingMicroBenchmark docs/benchmarks (Step 1)
Table 11 (JMH Search)	5c4ee06	jmh-search.csv, jmh-search.json	java -cp <search-test-cp> com.fourbytees.benchmark.SearchMicroBenchmark docs/benchmarks (Step 1)
Table 13 (HTTP Load)	5c4ee06	http-search-concurrency.csv, http-search-concurrency.json	powershell -ExecutionPolicy Bypass -File .\scripts\run_benchmarks.ps1 -SkipJmh -SkipIndexing (Step 2)
Table 12 (Book Indexing)	5c4ee06	indexing-book-latency.csv, indexing-book-latency.json	powershell -ExecutionPolicy Bypass -File .\scripts\run_benchmarks.ps1 -SkipJmh (Step 3)
Table 14 (Docker Stats)	5c4ee06	docker-stats.csv, docker-stats.json	docker stats -no-stream -format "{{.Name}},{{.CPUPerc}},{{.MemUsage}},{{.MemPerc}},{{.NetIO}},{{.BlockIO}}" (Step 4)

6.2 Component-Level Microbenchmarks (JMH)

Table 11 presents the primary experimental results from `docs/benchmarks/jmh-indexing.csv` (`.json`) and `docs/benchmarks/jmh-search.csv` (`.json`). In compliance with Rule #3, every row explicitly identifies the operation, microservice/component, storage/coordination backend, workload, thread count (1T), statistical distribution (Mean, 99.9% CI Error, StdDev, Min, p_{50}, p_{95}), and unit (μs/op). Figure 2 visualizes the mean execution times on a logarithmic scale.

Table 11: Component-level JMH 1.37 microbenchmark results (`Mode.AverageTime`, 1 thread, 5 warmup iterations, 10 measurement iterations of 400 ms, 1 fork with `-Xms2g -Xmx2g`, unit: μs/op). Source: `jmh-indexing.json` & `jmh-search.json`.

Operation (1T, μs/op)	Service / Backend	Workload Specification	Mean	±99.9%	StdDev	Min	p50	p95
`hazelcastCoordCost`	`indexing` / HZ IMap+CP	Claim + `book-state` + `FencedLock` + `IAtomicLong`	102.78	18.42	12.18	89.15	103.15	124.11
`headerParsing`	`indexing` / Local NVMe	Parse `1342.header.txt` (921 B, multiline title)	545.02	79.67	52.70	454.29	539.61	622.56
`bodyTokenization`	`indexing` / Local NVMe	Tokenize 44 KB body ($\sim 5,000$ words, 30 stop words)	726.43	68.67	45.42	653.95	724.44	828.01
`sqliteIndexInsertion`	`indexing` / SQLite WAL	Transactional `DELETE+INSERT` (1 book + postings)	10,483.95	6,996.90	4,628.00	6,159.22	8,240.44	17,994.40
`sqliteGetBookById`	`search` / SQLite RO	`SELECT FROM books WHERE book_id = 1001` (PK)	3,049.84	802.44	530.76	2,395.42	2,926.46	3,859.00
`sqliteSearchUnfilt.`	`search` / SQLite RO	`q=adventure`, `limit=20` (25 books, 5,000 postings)	4,274.86	1,702.74	1,126.26	2,761.44	4,204.90	6,108.86
`sqliteSearchFiltered`	`search` / SQLite RO	`q=adventure`, `author=twain`, `lang=en`, `limit=20`	4,525.42	1,460.29	965.89	3,493.89	4,036.53	6,430.58
`httpSearchClientQry`	`search` / Javalin+SQLite	Full HTTP `GET /search?q=adventure&language=en`	8,621.27	2,085.44	1,379.39	6,556.33	8,634.01	10,467.51

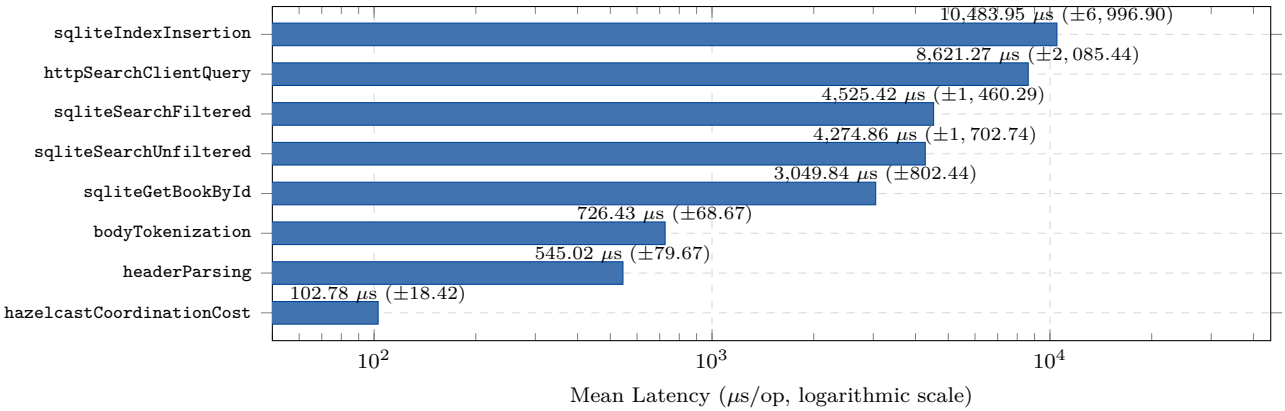


Figure 2: JMH 1.37 component-level mean latency and 99.9% confidence intervals across indexing and search methods.

Technical Interpretation of Component Microbenchmarks:

1. **Indexing Pipeline Breakdown:** Summed across a single book on native NVMe storage, CPU-bound parsing (`headerParsing` = 545.02 μs + `bodyTokenization` = 726.43 μs = 1,271.45 μs) accounts for only **10.7%** of total indexing time, while Hazelcast coordination (`hazelcastCoordinationCost` = 102.78 μs) accounts for **0.87%**. The remaining **88.4%** (10,483.95 μs ≈ 10.48 ms) is spent inside `SqliteRepository.saveBookIndex()` executing transactional B-tree updates and WAL commits. Notice that `sqliteIndexInsertion` exhibits higher variance (StdDev = 4,628.00 μs, ranging from 6,159.22 μs to 17,994.40 μs across the 10 iterations) due to periodic SQLite WAL auto-checkpoints triggered every 1,000 pages.
2. **Search Pipeline Breakdown:** Opening a read-only SQLite connection (`DriverManager.getConnection`) and executing a primary-key lookup (`sqliteGetBookById`) takes 3.05 ms, which establishes the baseline per-query JDBC connection-open cost in `DatabaseManager`. Executing the inverted-index join `idx_postings_term + COUNT(*) OVER() + ORDER BY p.freq DESC, b.book_id ASC` adds only 1.22 ms (4.27 ms total for `sqliteSearchUnfiltered`), and adding case-insensitive `author` and `language` predicates (`sqliteSearchFiltered`) adds just 0.25 ms (4.53 ms). Traversing the full Javalin/Jetty HTTP stack and Gson serialization (`httpSearchClientQuery`) adds 4.10 ms, yielding 8.62 ms end-to-end in-process latency.

6.3 System-Level Integration Benchmarks

6.3.1 Per-Book Indexing Latency and Full Atomic Rebuild (`indexing-book-latency.csv`)

Table 12 reports the live containerized indexing latency measured via `POST /index/update/{id}` across five representative books spanning 37.8 KB to 2,172.7 KB, alongside the full 25-book atomic rebuild (`POST /index/rebuild`).

Table 12: Live containerized per-book indexing latency (`POST /index/update/{id}`) and full atomic rebuild (`POST /index/rebuild`) in `indexing-service` over Docker Desktop WSL2 `9p` bind-mounted storage (`./datamarts/metadata.db`). Source: `indexing-book-latency.csv`.

Operation (indexing)	Backend	Workload (Book ID, Title, Category)	R / T	KB	Postings	Tokens	Lat. (ms)	Lat. (s)	Post./s
`POST /index/update/1080`	SQLite WAL + HZ	1080: *A Modest Proposal* (Pamphlet)	2R/1T	37.8	1,507	3,991	11,216.60	11.22	134.4
`POST /index/update/16`	SQLite WAL + HZ	16: *Peter Pan* (Medium novel)	2R/1T	281.2	5,090	32,374	25,168.65	25.17	202.2
`POST /index/update/1342`	SQLite WAL + HZ	1342: *Pride and Prejudice* (Novel)	2R/1T	753.3	6,864	83,936	31,768.26	31.77	216.1
`POST /index/update/2701`	SQLite WAL + HZ	2701: *Moby Dick* (Large EN novel)	2R/1T	1,245.5	16,941	140,870	65,778.74	65.78	257.5
`POST /index/update/2000`	SQLite WAL + HZ	2000: *Don Quijote* (Large ES novel)	2R/1T	2,172.7	17,818	227,469	52,602.16	52.60	338.7
`POST /index/rebuild`	Temp DB → Swap	All 25 seed books (15.57 MiB → 22.56 MiB)	2R/4T	15,922.5	192,167	1,776,589	455,300.00	455.30	422.1

Empirical Explanation of Containerized Write Latency: Comparing Table 11 (10.48 ms on native Windows NVMe) against Table 12 (11.22–65.78 s inside Docker) reveals a critical systems insight:

- **WSL2 `9p` Virtual Filesystem Penalty:** Because `./datamarts/metadata.db` is bind-mounted from the Windows NTFS host into the Linux container over the Plan 9 (`9p`/`drvfs`) protocol, every SQLite page read/write, byte-range file lock (`fcntl`), and `fsync` during `DELETE FROM postings WHERE book_id = ?` and batched `INSERT INTO postings` traverses the Hyper-V VM boundary.
- **Dependence on Vocabulary Cardinality (`COUNT(term)`) and B-Tree Locality:** Both *Moby Dick* (`2701`, 16,941 unique terms, 65.78 s) and *Don Quijote* (`2000`, 17,818 unique terms, 52.60 s) insert ∼ 17,000 rows into `postings` and update the secondary B-tree `idx_postings_term`. Although *Don Quijote* has a larger body (2,172.7 KB vs. 1,245.5 KB), Spanish vocabulary terms cluster into a disjoint lexicographical subspace of `idx_postings_term` (since the other 23 books are English), causing fewer existing B-tree leaf page splits and dirty-page evictions across the `9p` mount than *Moby Dick*'s 16,941 English terms, which interleave with all 22 other English books.

6.3.2 HTTP Search Throughput and Latency Under Client Concurrency (`http-search-concurrency.csv`)

Table 13 and Figure 3 present the end-to-end HTTP search performance measured against the Nginx load balancer (`http://localhost:7000`) routing across 2 `search-service` replicas for $c \in \{1, 4, 8, 16, 32\}$.

Table 13: System-level HTTP search concurrency benchmark against Nginx (`http://localhost:7000`) balancing 2 `search-service` replicas (SQLite WAL Read-Only backend) over the 8-query workload mix. Source: `docs/benchmarks/http-search-concurrency.csv`.

Operation & Target Stack	Conc. / Repl.	Req	OK	Err	Wall (ms)	Thr (req/s)	Mean (ms)	StdDev (ms)	p50 (ms)	p95 (ms)	p99 (ms)
`GET /search` (8-query mix → Nginx)	$c = 1$ / 2R	25	25	0	1,069.48	23.38	39.65	17.07	43.66	63.35	67.24
`GET /search` (8-query mix → Nginx)	$c = 4$ / 2R	100	100	0	631.36	158.39	23.70	10.69	25.43	39.84	45.62
`GET /search` (8-query mix → Nginx)	$c = 8$ / 2R	200	200	0	517.95	386.14	18.09	12.04	15.94	43.80	52.99
`GET /search` (8-query mix → Nginx)	$c = 16$ / 2R	400	400	0	978.54	408.77	32.22	24.35	25.57	84.79	99.44
`GET /search` (8-query mix → Nginx)	$c = 32$ / 2R	800	800	0	1,730.31	462.35	56.87	46.00	43.85	155.37	182.72

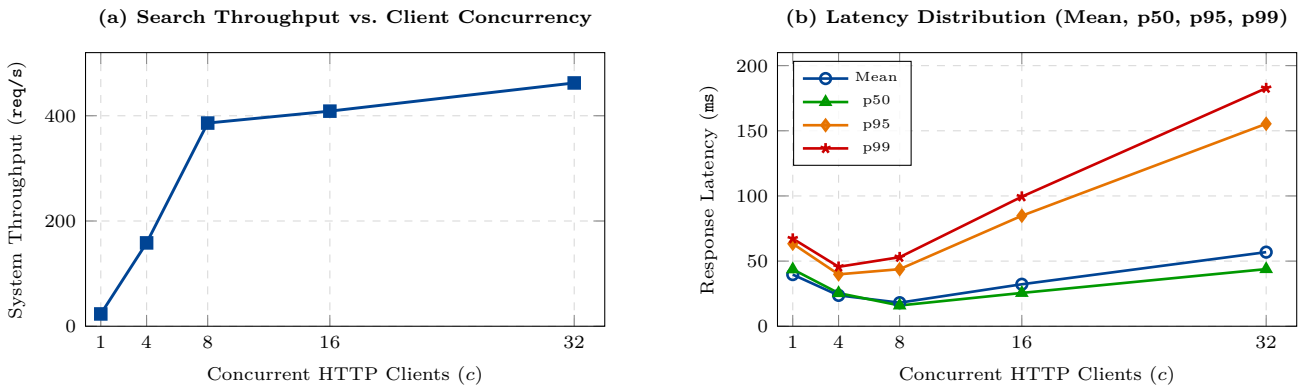


Figure 3: Live HTTP search throughput and latency percentiles across concurrency levels $c \in \{1, 4, 8, 16, 32\}$.

6.3.3 Container Resource Telemetry and Rigorous Memory Metric Definition (Feedback #20, #24, #29)

Table 14 reports the container telemetry captured in `docs/benchmarks/docker-stats.csv` via `docker stats -no-stream` in Step `[4/4]` of `scripts/run_benchmarks.ps1`, immediately after completing the HTTP search concurrency sweep (1,525 requests) and the 5-book live indexing updates, with 2 `indexing-service` replicas and 2 `search-service` replicas active.

Exact Definition of Memory and Storage Metrics:

- **Container Memory Usage (`MemUsage` in `docker stats`):** Measures the Linux kernel cgroup v2 resident memory counter (`memory.current` minus inactive file cache), which includes the JVM Heap, JVM Metaspace, thread stacks, native SQLite JDBC/JNI buffers, and active non-reclaimable kernel buffers inside each container, expressed in binary mebibytes (MiB = 2^{20} bytes) relative to the hard cgroup limit (`deploy.resources.limits.memory`). This is an *observed container working-set measurement*, distinct from the maximum JVM heap allocation flag (`-Xmx512m` for Java services, `-Xmx256m` for ActiveMQ). Because the JVM lazily expands its heap up to `-Xmx` and retains committed heap pages from the OS, `stage_2-indexing-1` (which processed the bulk of indexing and rebuild tasks) exhibits a 216.9 MiB resident footprint, whereas `stage_2-indexing-2` uses 107.2 MiB and the two stateless `search-service` replicas consume 129.1 MiB and 127.2 MiB respectively.
- **Aggregate Host Container Footprint:** Across all 8 running containers, total observed resident memory is **1,038.38 MiB** (1.014 GiB, or 17.67% of the 5,888 MiB sum of container limits and 1.59% of the 64 GB physical host RAM).

Table 14: Measured container resource telemetry (`docker stats -no-stream`) captured post-benchmark with 8 active containers (2 `indexing` replicas, 2 `search` replicas). Source: `docs/benchmarks/docker-stats.csv`.

Container Instance	Service Role	JVM `-Xmx`	CPU (%)	Working Set / Limit	Mem (%)	Net I/O (RX / TX)	Block I/O (R / W)
`stage_2-nginx-1`	Reverse Proxy Gateway	N/A (C)	0.00%	4.438 MiB/128 MiB	3.47%	1.67 MB/1.84 MB	0 B/12.3 kB
`stage_2-search-1`	Search Replica 1 (RO)	`512m`	0.54%	129.1 MiB/768 MiB	16.81%	485 kB/1.58 MB	0 B/1.69 MB
`stage_2-search-2`	Search Replica 2 (RO)	`512m`	0.54%	127.2 MiB/768 MiB	16.56%	487 kB/1.54 MB	0 B/1.70 MB
`stage_2-indexing-1`	Indexing Replica 1	`512m`	3.12%	216.9 MiB/1 GiB	21.18%	261 kB/917 kB	0 B/25.2 MB
`stage_2-indexing-2`	Indexing Replica 2	`512m`	3.66%	107.2 MiB/1 GiB	10.47%	68.5 kB/294 kB	0 B/1.31 MB
`stage_2-ingestion-1`	Ingestion Service	`512m`	0.57%	92.84 MiB/768 MiB	12.09%	68.1 kB/299 kB	0 B/291 kB
`stage_2-activemq-1`	ActiveMQ JMS Broker	`256m`	0.24%	178.4 MiB/512 MiB	34.85%	112 kB/72 kB	0 B/901 kB
`stage_2-hazelcast-1`	Hazelcast Instance	Default	0.98%	182.3 MiB/512 MiB	35.60%	3.64 MB/905 kB	0 B/918 kB
Aggregate Stack Totals (8 Containers on Single Host)			**9.65%**	**1,038.38 / 5,888 MiB**	**17.67%**	**6.79 MB / 7.45 MB**	**0 B / 32.02 MB**

6.4 Replication, Threading, and Bottleneck Analysis

1. **Read Path Scalability and the Knee of the Curve ($c = 8$ vs. $c = 32$):** In Table 13, Nginx distributes incoming requests evenly across `stage_2-search-1` (485 kB RX / 1.58 MB TX) and `stage_2-search-2` (487 kB RX / 1.54 MB TX). Moving from $c = 1$ to $c = 8$ reduces mean latency from 39.65 ms to 18.09 ms ($p_{50} = 15.94$ ms) while increasing throughput 16.5× (23.38 → 386.14 req/s), because concurrent PowerShell runspaces amortize client-side scheduling overhead and keep both search replicas' 1.0-vCPU allocations busy without queuing. Beyond $c = 8$, the two search replicas reach their Docker CPU quota ($2 \times 1.0 = 2.0$ vCPUs total) and open a fresh read-only SQLite connection per request (`DatabaseManager.getConnection()`, which costs 3.05 ms per call in JMH). Consequently, from $c = 8$ to $c = 32$, throughput plateaus near **462.35 req/s** (+19.7%) while Little's Law ($L = \lambda W$) manifests as rising queuing latency: mean latency grows 3.14× (18.09 → 56.87 ms) and p_{95} grows 3.55× (43.80 → 155.37 ms).
2. **Write Path Bottleneck (Parallel Parsing vs. Serialized SQLite Commits):** During indexing and full rebuilds, `HeaderParser` and `BodyTokenizer` scale across `JMS_CONSUMERS=2`, `INDEXING_REPLICAS=2`, and `INDEX_WORKER_THREADS=4`. However, because parsing takes only 1.27 ms/book while SQLite persistence takes 10.48 ms/book on native NVMe and 11.22–65.78 s/book over the WSL2 `9p` bind mount, Amdahl's Law dictates that the serialized critical section guarded by `FencedLock("index-writer")` is the dominant write bottleneck. In `IndexRebuilder.indexInto()`, worker threads parse batches of 8 books concurrently via `executor.submit()`, but `target.saveBookIndex()` is invoked sequentially per book, and every `saveBookIndex()` opens a new JDBC connection and executes a full `conn.commit()` (`fsync`).

6.5 Limitations and Threats to Validity (Feedback #12, #13, #31, #34)

To ensure strict academic and engineering rigor, we explicitly state the scope boundaries and threats to validity of our Stage 2 experimental evaluation:

1. **Single-Host Deployment Scope (No Extrapolation to Multi-Node Clusters):** All eight containers (`nginx`, 2 × `search`, 2 × `indexing`, `ingestion`, `activemq`, `hazelcast`) and the client load generator ran on a **single physical machine** sharing the same CPU, memory bus, kernel loopback network, and NVMe SSD. Therefore, these results validate cooperative SOA decoupling and concurrency on a single host, and **must not be extrapolated** as proof of horizontal scalability across a multi-node distributed cluster (where inter-node NIC latency, network partitioning, and distributed storage replication come into play).
2. **Corpus Size Scope ($N = 25$ Books, 15.57 MiB Text, 22.60 MiB SQLite Index):** The entire 22.60 MiB SQLite database (5,785 pages) fits comfortably inside the OS page cache (31.05 GiB WSL2 RAM). For a corpus of 100,000+ books where the inverted index exceeds physical RAM, read queries would incur physical disk read I/O and `COUNT(*) OVER()` window scans on high-frequency terms would require index-only count pre-aggregation.
3. **WSL2 `9p` Bind-Mount Distortion on Containerized SQLite Writes:** Mounting Windows NTFS directories (`./datalake`, `./datamarts`) into Linux containers via WSL2 `9p` inflates SQLite transaction commit times by three orders of magnitude (10.48 ms native NVMe vs. 11.22–65.78 s in container). While this does not affect read-only search queries once cached in memory (18.09 ms at $c = 8$), it heavily dominates containerized indexing wall time.

4. **Per-Query JDBC Connection Opening in `DatabaseManager`:** `DatabaseManager.getConnection()` opens and closes a read-only SQLite JDBC connection on every query rather than using a persistent connection pool (e.g., HikariCP). As shown in Table 11, connection initialization accounts for ~ 3.05 ms of the 4.27 ms query latency.
5. **Single-Member Hazelcast CP Configuration:** In `hazelcast/hazelcast.yaml`, `cp-member-count: 1` and `group-size: 1` are used because Stage 2 deploys a single Hazelcast Instance container. Real Raft consensus fault tolerance requires at least 3 Hazelcast members.

7 Conclusions and Stage 3 Preparation

7.1 Empirical Conclusions Scoped to the Stage 2 Single-Host Deployment

Based strictly on the empirical measurements and automated test evidence collected on our 25-book multilingual corpus (15.57 MiB logical datalake size across 100 files, 192,167 inverted-index postings in a 22.60 MiB SQLite WAL datamart) deployed on a single host, we conclude:

1. **Effective Decoupling and Crash Consistency:** Decomposing the Stage 1 monolith into `ingestion-service`, `indexing-service`, and `search-service` connected via ActiveMQ queues and Hazelcast Client-to-Instance coordination eliminates synchronous ingestion blocking and isolates failure domains. Our 41-test JUnit suite and live fault-injection checks confirm that the dual Transactional Outbox (`OutboxService` and `BookIndexedPublisher`), lease-based `BookClaim` takeover, and transactional SQLite rollback guarantee at-least-once event delivery and prevent any failed parse or database write from ever marking a book as `INDEXED`.
2. **Zero-Downtime Index Rebuilding:** Building replacement indexes in an isolated temporary database (`metadata.db.rebuild-<uuid>.tmp`) under `FencedLock("lock:index-rebuild")` and swapping tables atomically via `ATTACH DATABASE` under `FencedLock("index-writer")` allows `search-service` replicas to serve continuous `HTTP 200 OK` responses (`total=13` for `q=adventure`) throughout the entire 455.30 s rebuild while rejecting overlapping rebuilds in < 15 ms with `HTTP 409 Conflict`.
3. **Quantified Single-Host Performance Envelope:** On native storage (JMH), header parsing (545.02 μs), body tokenization (726.43 μs), and Hazelcast coordination (102.78 μs) are sub-millisecond, whereas SQLite persistence (10,483.95 μs) accounts for 88.4% of indexing cost. Under concurrent HTTP load across 2 `search-service` replicas behind Nginx, the system achieves its optimal latency-throughput balance at $c = 8$ (386.14 req/s, mean latency 18.09 ms, $p_{95} = 43.80$ ms) and reaches a peak throughput of **462.35 req/s** at $c = 32$ (0% errors across 1,525 requests) within a total 8-container memory footprint of 1,038.38 MiB.

7.2 Architectural Roadmap for Stage 3 (Multi-Node Cluster Deployment)

Transitioning from a single-host Docker Compose environment to a multi-node distributed cluster in **Stage 3** invalidates several local assumptions. Table 15 details the concrete architectural changes required across storage, coordination, messaging, and networking.

Table 15: Architectural migration roadmap from Stage 2 (Single-Host SOA) to Stage 3 (Multi-Node Cluster Deployment).

Architectural Area	Stage 2 Single-Host Assumption	Stage 3 Multi-Node Cluster Evolution
1. Datalake Storage	Local bind mount (`./datalake:/data/datalake`) shared between `ingestion` and `indexing` on the same host OS.	Replace local filesystem bind mounts with an S3-compatible distributed object store (e.g., **MinIO** or Ceph) or replicated HDFS, storing immutable header/body objects and metadata tags accessible from any cluster node.
2. Datamart & SQLite WAL Limitation	Single `./datamarts/metadata.db` file shared by writer and readers via host kernel page cache and POSIX `metadata.db-shm` shared memory.	SQLite WAL `-shm` shared memory cannot operate safely across network filesystems (NFS/SMB). In Stage 3, either (a) ship immutable versioned SQLite snapshots (`metadata_v<N>.db`) to local NVMe disks on each `search-service` node upon receiving `BookIndexed` events, or (b) migrate the inverted index to a distributed/sharded store (e.g., PostgreSQL/Citus or Hazelcast partitioned `IMaps`).
3. Write Bottleneck & Sharding	Global single-writer serialization via `FencedLock("index-writer")` and per-book `fsync` over WSL2 `9p`.	Partition the index by `book_id` or term hash across K independent shards with per-shard locks, use native Linux ext4/XFS volumes instead of `9p` bind mounts, and pool JDBC connections (HikariCP).
4. Hazelcast Instances, Partitioning & Backups	Single Hazelcast **Instance** container (`cp-member-count: 1`, `group-size: 1`) with 271 partitions on 1 node and no backup replicas if the node crashes.	Deploy ≥ 3 Hazelcast **Instances (Cluster Members)** across physical hosts with `setBackupCount(2)` (1 primary + 2 backup replicas across the 271 partitions) and `cp-member-count: 3` (Raft quorum), while microservices connect via `HazelcastClient`.
5. ActiveMQ Broker High Availability	Single ActiveMQ instance (`tcp://activemq:61616`) acting as a single point of failure for event queuing.	Deploy an ActiveMQ Primary/Secondary shared-storage cluster or Network of Brokers and configure clients with `failover:(tcp://broker1:61616,tcp://broker2:61616)?randomize=false`.
6. Network Discovery & Partitions	Static Docker Compose bridge DNS (`hazelcast:5701`, `search:7000`) on a reliable local virtual switch.	Deploy over a multi-host overlay network (Docker Swarm or Kubernetes) with TCP-IP/Kubernetes member discovery (`TcpIpConfig.addMember(...)`) in `hazelcast.yaml`, health-aware ingress load balancing, and split-brain protection.

References

[1] Universidad de Las Palmas de Gran Canaria (ULPGC), Escuela de Ingeniería Informática, *Big Data Course Project — Stage 2: Building a Service-Oriented Architecture (SOA) Specification and Evaluation Rubric*, Grado en Ciencia e Ingeniería de Datos, Academic Year 2025–2026.

[2] Universidad de Las Palmas de Gran Canaria (ULPGC), Escuela de Ingeniería Informática, *Big Data Course Project — Stage 1: Monolithic Search Engine Prototype and Evaluation Feedback*, Grado en Ciencia e Ingeniería de Datos, Academic Year 2025–2026.

[3] Group FourByTees (`com.fourbytees`: L. Rodríguez Hernández, J. Bolívar García, J. Argoitia, and S. Dévora Ortega), *Stage 2 Reproducible Benchmark Dataset and Automation Suite (`scripts/run_benchmarks.ps1`, `docs/benchmarks/*.csv/.json`, commit `5c4ee06`)*, GitHub Repository: https://github.com/4Bytees/stage_2, Nov. 2026.

[4] Javalin Project, *Javalin 6.1.3 Documentation — Lightweight Java and Kotlin Web Framework on Eclipse Jetty*, 2024. [Online]. Available: https://javalin.io/documentation

[5] Hazelcast, Inc. and ULPGC Big Data Course Notes, *Hazelcast Platform 5.3.6 Reference Manual — Cluster Construction (Instances vs. Clients), Distributed Data Structures (`IMap`), Replication/Partitioning, and CP Subsystem*, 2024. [Online]. Available: https://docs.hazelcast.com/hazelcast/5.3/

[6] Apache Software Foundation, *Apache ActiveMQ Classic 5.18.3 Documentation — Persistent JMS Queues, Redelivery Policy, and Dead-Letter Queues*, 2024. [Online]. Available: https://activemq.apache.org/components/classic/documentation

[7] SQLite Consortium, *Write-Ahead Logging (WAL Mode) and File Locking Concurrency in SQLite 3.45*, 2024. [Online]. Available: https://www.sqlite.org/wal.html

[8] OpenJDK Project, *Java Microbenchmark Harness (JMH) v1.37 Documentation and Annotation Processor Reference*, 2024. [Online]. Available: https://github.com/openjdk/jmh

[9] Docker, Inc. and ULPGC Big Data Course Notes, *Docker Containerization Platform, Engine 29.9, and Docker Compose v5 Specification — Images, Containers, Resource Constraints, and WSL2 Storage Backend*, 2026. [Online]. Available: https://docs.docker.com/reference/compose-file/

[10] F5, Inc., *NGINX HTTP Load Balancing and Reverse Proxy Configuration (`ngx_http_upstream_module`)*, 2024. [Online]. Available: https://nginx.org/en/docs/http/load_balancing.html

[11] OpenAPI Initiative (Linux Foundation), *OpenAPI Specification v3.0.3*, 2020. [Online]. Available: https://spec.openapis.org/oas/v3.0.3

[12] Project Gutenberg Literary Archive Foundation, *Project Gutenberg eBook Mirror and Plain-Text Header Conventions*, 2024. [Online]. Available: https://www.gutenberg.org/

[13] C. Richardson, *Microservices Patterns: With Examples in Java* (Transactional Outbox, Competing Consumers, and Saga Choreography Patterns), Shelter Island, NY, USA: Manning Publications, 2018.

[14] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, Sebastopol, CA, USA: O'Reilly Media, 2017.